Zelta MCP Server Implementation Plan

Version: 1.0 **Date:** January 2026 **Author:** Infrastructure Team

---

Executive Summary

This document outlines the implementation plan for building an MCP (Model Context Protocol) server for Zelta, enabling AI assistants to perform intelligent storage management operations. The integration leverages Zelta's unique strengths: non-destructive operations, storage efficiency through ZFS block sharing, and hybrid cloud support.

Strategic Differentiation

Unlike generic Kubernetes MCP wrappers, Zelta+MCP provides:

Capability	Traditional Orchestrators	Zelta+MCP
Data Model	Pods, containers (ephemeral)	Datasets, snapshots, replication state
Operations	Delete and recreate	Clone, test, rotate when ready
Cost Model	N copies = N × storage	N clones = shared blocks + deltas
Scope	Single cloud vendor	Any ZFS system (bare metal to cloud)

---

1. Architecture Overview

1.1 System Architecture

Layer 1: AI Application (Claude, ChatGPT, etc.)

- Connects via MCP Protocol (JSON-RPC 2.0)

Layer 2: Zelta MCP Server

- **Tools** - Actions (backup, clone, rotate, revert, match)
- **Resources** - Data (config, status, snapshots)
- **Prompts** - Templates (disaster recovery, test environment)
- **Executor** - Runs Zelta commands, parses JSON output

Layer 3: Zelta CLI (`bin/zelta` → `share/zelta/*.awk`)

- Connects via SSH for remote endpoints

Layer 4: ZFS Endpoints

- Local pools, remote servers, cloud instances

1.2 Transport Strategy

Transport	Use Case	When to Use
STDIO	Local CLI integration	Claude Code, local AI tools

HTTP	Centralized management	Multi-user, web dashboards

Phase 1: STDIO only (simplest, matches Zelta's local-first design) **Phase 2:** Add HTTP transport for centralized management scenarios

---

2. Tool Design

2.1 Core Tools (Phase 1)

Based on MCP best practices ("less is more"), we expose 8 focused tools:

Read-Only Tools (Intelligence Gathering)

Tool	Description	Maps to
`zelta_match`	Compare source/target datasets, report sync status	`zelta match --json`
`zelta_list`	List datasets with properties (size, snapshots, etc.)	`zfs list` wrapper
`zelta_status`	Get replication status for a policy or endpoint	`zelta match` + parsing

Action Tools (Safe Operations)

Tool	Description	Maps to	Safe?
`zelta_backup`	Replicate datasets	`zelta backup`	Yes
`zelta_snapshot`	Create snapshot	`zelta snapshot`	Yes
`zelta_clone`	Clone dataset	`zelta clone`	Yes
`zelta_rotate`	Recover sync continuity	`zelta rotate`	Yes
`zelta_revert`	Non-destructive rollback	`zelta revert`	Yes

Policy Tools (Phase 2)

Tool	Description	Maps to
`zelta_policy_list`	List configured backup policies	Config parsing
`zelta_policy_run`	Execute a named policy	`zelta policy`
`zelta_policy_status`	Check status of all policy targets	`zelta match` loop

2.2 Tool Specifications

`zelta_match` - Dataset Comparison

```
{ "name": "zelta_match", "description": "Compare source and target datasets to identify sy
```

`zelta_backup` - Dataset Replication

```
{ "name": "zelta_backup", "description": "Replicate ZFS datasets from source to target witl
```

`zelta_clone` - Space-Efficient Cloning

```
{ "name": "zelta_clone", "description": "Create a space-efficient clone of a dataset at a s
```

`zelta_rotate` - Sync Recovery

```
{ "name": "zelta_rotate", "description": "Recover sync continuity when source and target ha
```

`zelta_revert` - Non-Destructive Rollback

```
{ "name": "zelta_revert", "description": "Rewind a dataset to a previous snapshot WITHOUT d
```

---

3. Resources Design

MCP Resources expose data for AI context without side effects.

3.1 Resource URIs

URI Pattern	Description	Content Type
`zelta://config`	Current zelta.conf contents	text/plain
`zelta://env`	Current zelta.env defaults	text/plain
`zelta://status/{endpoint}`	Backup status for endpoint	application/json
`zelta://snapshots/{dataset}`	Snapshot list for dataset	application/json
`zelta://policies`	List of configured policies	application/json

3.2 Resource Implementation

```
@mcp.resource("zelta://config") def get_config() -> str: """Current Zelta policy configurat
```

---

4. Prompts Design

Pre-built instruction templates for common workflows.

4.1 Disaster Recovery Prompt

```
@mcp.prompt() def disaster_recovery_assessment(source: str, target: str) -> list: """Genera
```

4.2 Test Environment Prompt

```
@mcp.prompt() def create_test_environment(production_dataset: str) -> list: """Guide creat
```

4.3 Compliance Audit Prompt

```
@mcp.prompt() def compliance_audit(policy_name: str) -> list: """Audit backup compliance fo
```

---

5. Implementation Phases

Phase 1: Core Foundation (Week 1-2)

Goal: Working MCP server with read-only tools

Deliverables:

- [ ] Project scaffolding (pyproject.toml, directory structure)
- [ ] `zelta_match` tool implementation
- [ ] `zelta_list` tool implementation
- [ ] `zelta_status` tool implementation
- [ ] Basic resource: `zelta://config`
- [ ] STDIO transport working
- [ ] Unit tests for all tools
- [ ] Claude Desktop integration tested

Directory Structure:

- `zelta-mcp/`
  - `pyproject.toml`
  - `README.md`
  - `src/zelta_mcp/`
    - `__init__.py`
    - `server.py` - FastMCP server entry point
    - `executor.py` - Zelta command execution
    - `tools/` - match.py, list.py, status.py
    - `resources/` - config.py
  - `tests/` - test_match.py, conftest.py

Phase 2: Action Tools (Week 3-4)

Goal: Safe write operations

Deliverables:

- [ ] `zelta_backup` tool
- [ ] `zelta_snapshot` tool
- [ ] `zelta_clone` tool
- [ ] `zelta_rotate` tool
- [ ] `zelta_revert` tool
- [ ] Dry-run mode for all action tools
- [ ] Integration tests with real ZFS
- [ ] Error handling and user-friendly messages

Phase 3: Policy & Resources (Week 5-6)

Goal: Policy orchestration and rich resources

Deliverables:

- [ ] `zelta_policy_list` tool
- [ ] `zelta_policy_run` tool
- [ ] `zelta_policy_status` tool
- [ ] Resource: `zelta://snapshots/{dataset}`
- [ ] Resource: `zelta://status/{endpoint}`
- [ ] Resource: `zelta://policies`
- [ ] Prompts: disaster recovery, test environment, compliance audit

Phase 4: Production Hardening (Week 7-8)

Goal: Production-ready release

Deliverables:

- [ ] HTTP transport option
- [ ] Authentication for HTTP mode
- [ ] Comprehensive documentation
- [ ] Performance optimization
- [ ] Logging and observability
- [ ] Package publishing (PyPI)
- [ ] Claude Desktop configuration examples

---

6. Code Implementation

6.1 Project Configuration

pyproject.toml:

```
[project] name = "zelta-mcp" version = "0.1.0" description = "MCP server for Zelta ZFS bacl
```

6.2 Server Implementation

src/zelta_mcp/server.py:

```
#!/usr/bin/env python3 """Zelta MCP Server — AI-powered ZFS backup management.""" import as
```

6.3 Claude Desktop Configuration

claude_desktop_config.json:

```
{ "mcpServers": { "zelta": { "command": "zelta-mcp", "args": [], "env": { "PATH": "/usr/lo
```

Or if running from source:

```
{ "mcpServers": { "zelta": { "command": "python", "args": ["-m", "zelta_mcp.server"], "cwd"
```

---

7. Testing Strategy

7.1 Unit Tests

```
# tests/test_match.py import pytest from unittest.mock import AsyncMock, patch from zelta_m
```

7.2 Integration Tests

```
# tests/test_integration.py import pytest import subprocess @pytest.fixture def test_pool()
```

---

8. Security Considerations

8.1 Authentication Model

Security Boundaries:

1. **MCP Client** (Claude) ↔ **Zelta MCP Server**
   - Transport: STDIO (local)
   - Server runs as invoking USER with USER's credentials
2. **Zelta MCP Server** → **Remote Endpoints**
   - Transport: SSH (user's keys)
   - ZFS delegation controls access on endpoints

8.2 ZFS Delegation

Zelta relies on ZFS delegation for fine-grained permissions:

```
# On remote endpoint, allow backup user specific permissions zfs allow -u backupuser send,s
```

8.3 MCP Server Permissions

The MCP server inherits the invoking user's permissions:

- SSH keys for remote access
- ZFS permissions on local pools
- No additional privilege escalation

---

9. Example Conversations

9.1 Disaster Recovery

User: "Check if my database backup is current"

AI uses: `zelta_match("db.prod.local:tank/postgres", "backup.local:tank/backups/db")`

AI Response: "Your PostgreSQL backup is 4 hours behind. There are 3 snapshots pending transfer totaling 2.1GB. The last successful sync was at 2026-01-21 08:00 UTC. Would you like me to run a backup now?"

9.2 Test Environment Creation

User: "I need to test a schema migration on a copy of production"

AI uses:

1. `zelta_snapshot("db.prod.local:tank/postgres", name="pre-migration-test")`
2. `zelta_clone("db.prod.local:tank/postgres@pre-migration-test", "db.prod.local:tank/postgres-test")`

AI Response: "Created test environment at tank/postgres-test. This clone shares 847GB of blocks with production and currently uses only 12MB of unique storage. You can run your migration safely - if it fails, simply destroy the clone. If it succeeds, we can use zelta_rotate to swap it into production while preserving the original."

9.3 Recovery from Divergence

User: "Someone accidentally wrote to the backup server, now syncs are failing"

AI uses: `zelta_match("prod:tank/data", "backup:tank/data")`

AI Response: "The backup has diverged from production. Last common snapshot was 3 days ago (@zelta_2026-01-18_00.00.00). I can use zelta_rotate to recover:

1. Rename diverged backup to tank/data_zelta_2026-01-18_00.00.00 (preserved)
2. Clone from common snapshot to tank/data
3. Sync 3 days of production changes

Both versions will be preserved. Want me to proceed?"

---

10. Success Metrics

10.1 Technical Metrics

Metric	Target	Measurement
Tool response time	< 2s for read-only	Logging
Backup throughput	Match native Zelta	Benchmarks
Error rate	< 1%	Monitoring
Test coverage	> 80%	pytest-cov

10.2 User Experience Metrics

Metric	Target	Measurement
Successful task completion	> 90%	User feedback
AI understanding accuracy	> 95%	Review conversations
Documentation clarity	Positive feedback	User surveys

---

11. Future Enhancements

Phase 5+: Advanced Features

1. **Streaming Progress** - Real-time transfer progress for long backups
2. **Cost Calculator** - Estimate storage costs across providers
3. **Policy Suggestions** - AI-driven backup schedule recommendations
4. **Anomaly Detection** - Alert on unusual backup patterns
5. **Multi-Site Orchestration** - Coordinate backups across regions

---

Appendix A: MCP Protocol Reference

JSON-RPC Message Format

Request:

```
{ "jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": { "name": "zelta_backup", "a
```

Response:

```
{ "jsonrpc": "2.0", "id": 1, "result": { "content": [ { "type": "text", "text": "{\"succes
```

Appendix B: Zelta Command Reference

Command	MCP Tool	Description
`zelta match`	`zelta_match`	Compare datasets
`zelta backup`	`zelta_backup`	Replicate datasets
`zelta snapshot`	`zelta_snapshot`	Create snapshots
`zelta clone`	`zelta_clone`	Create clones
`zelta rotate`	`zelta_rotate`	Recover from divergence
`zelta revert`	`zelta_revert`	Non-destructive rollback
`zelta policy`	`zelta_policy_run`	Execute policies

Appendix C: Resources

Official Documentation

- MCP Specification
- Python SDK
- FastMCP Tutorial

Zelta Documentation

- Zelta Manual
- Backup Operations
- Match Operations
- Rotate Operations

Document Version: 1.0 | Last Updated: January 2026